Information
Technology
Institute

Version Control

Git , Github

git

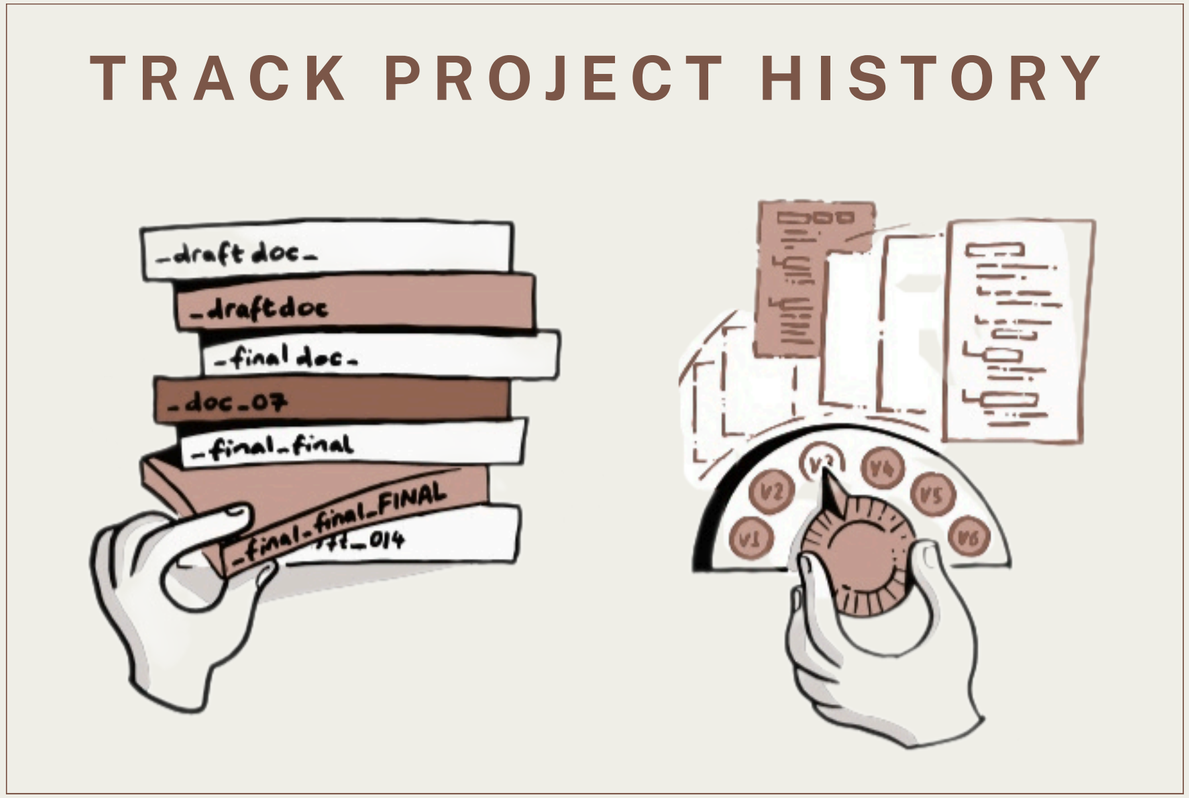
COURSE OVERVIEW

- What is Version Control?
- Why VCS is important?
- Types of VCS.
- Understand GIT , Git workflow
- Terms [Repository, commit, Branch , Merge , push ,Clone, ..etc]
- Github.
- Difference Between Git & Github.

WHAT IS VERSION CONTROL?

- Version Control System (VCS) is a system tracks changes, allows collaboration, and saves history

- that records changes to a file or set of files over time so that you can recall specific versions later



TOOLS

Git:

https://git-scm.com/

Github:

https://docs.github.com/

TYPES OF "VCS"

Information Technology Institute

VCS

Local Version Control System (LVCS)

Ex:Copy-paste folder

Centralized Version Control System (CVCS)

Ex: SVN, Perforce

Distributed Version Control System (DVCS)

Ex: Git,GitHub GitLab

UNDERSTAND GIT , GIT WORKFLOW

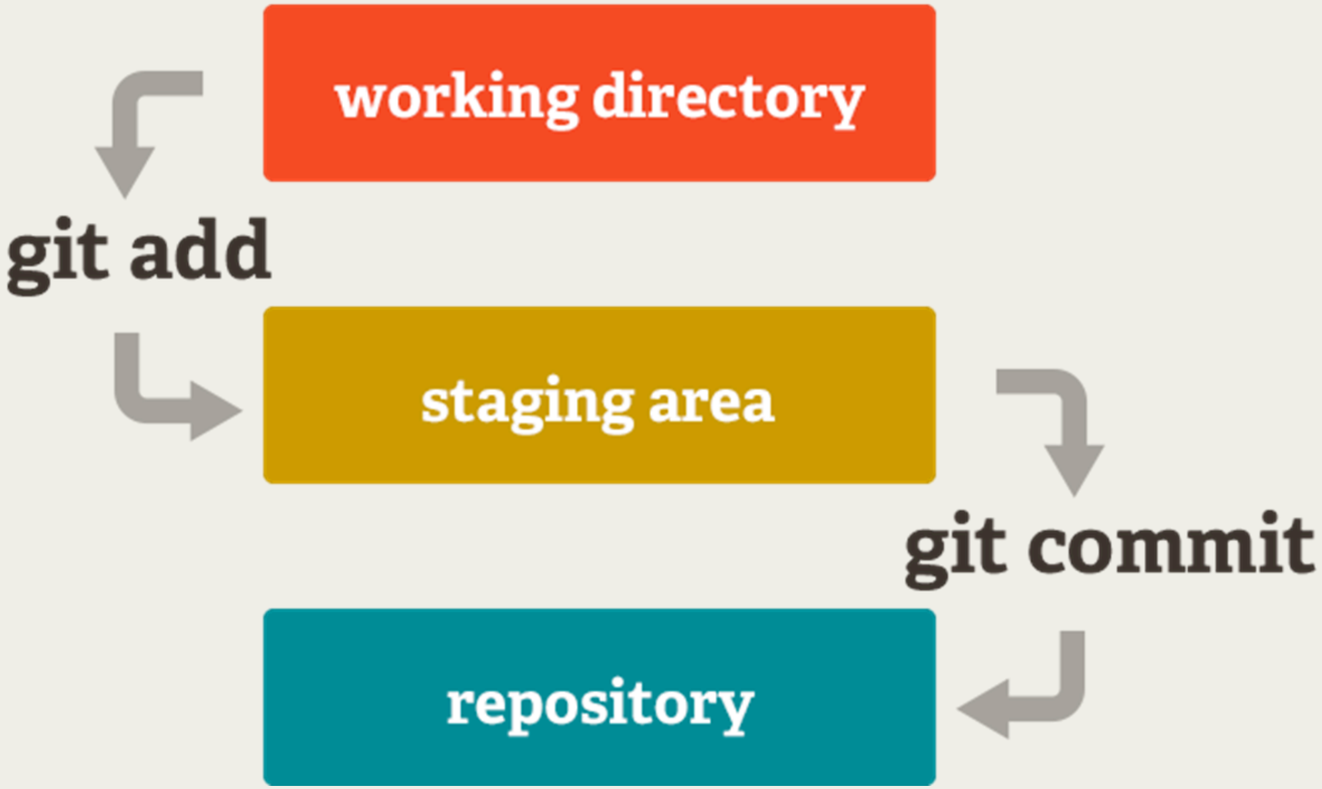
WHAT IS GIT??

- Distributed Version Control System
- Git tracks changes to source code

Git Areas:

working Directory	Where you edit files
Staging Area	Where you prepare changes (git add)
Repository	Where commits are stored (git commit)

GIT WORKFLOW



TERMS

🗂️ Repository (Repo)

A folder that Git tracks, Contains all your project files + version history.

Analogy: Like a DB with your project and all its past versions inside.

💾 Commit:

A snapshot of your project at a specific moment.

You write a message to describe what changed.

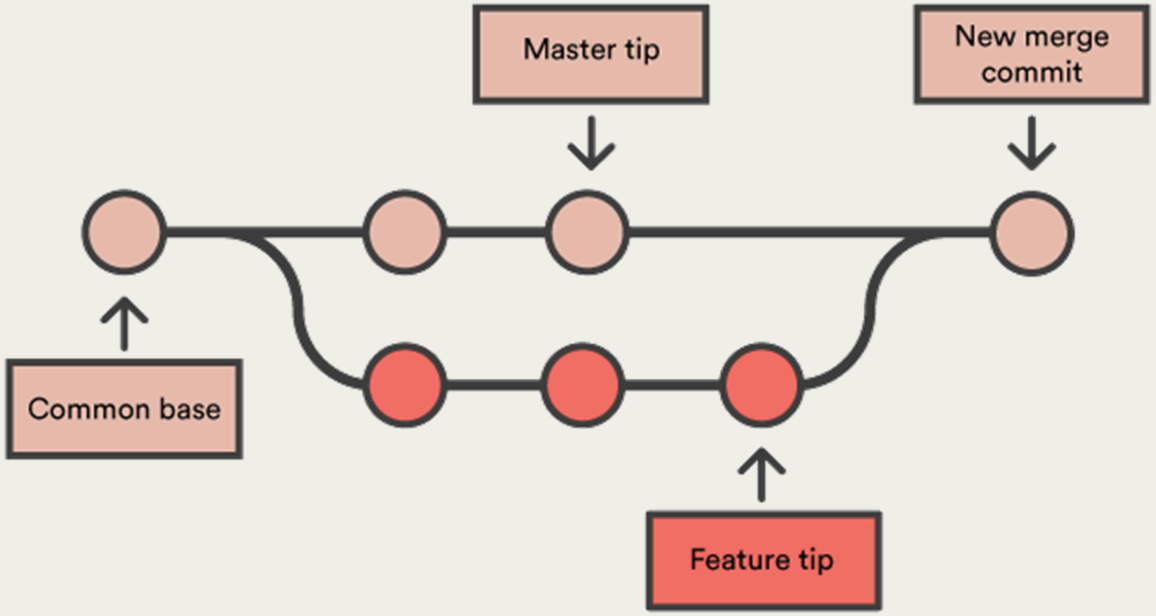
🌱 Branch:

A separate line of development from the main code.

Used to work on features or fixes without affecting the main code.

🔀 Merging

Merging is Git's way of putting a forked history back together again. The git merge command lets you take the independent lines of development created by git branch and integrate them into a single branch.



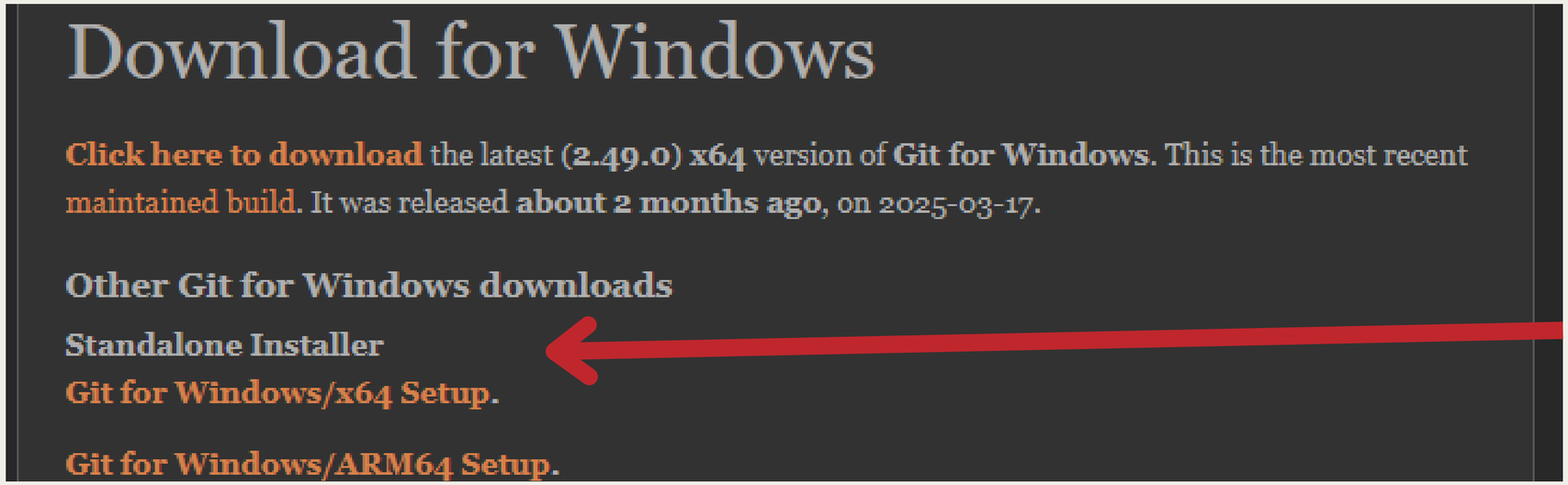
WHY "VCS" IS IMPORTANT?

1. Tracking a Project's History
2. Sync with multiple computers
3. Working in a team
4. Safely create and test new features

START DEMO

STEPS

1- Download Git from: https://git-scm.com/downloads/win
choose standalone Installer



2- Create your project folder and open terminal on it
use cmd or git bash or VS terminal

STEPS

3- Make sure you are login by commands:

```
git config --global user.name “your name”
git config --global user.email “your email”
```

4- Create repositery on your folder :

```
git init
```

this is important for native project like html,css
but not need on some frameworks like angular

GIT COMMANDS

git init : will do all of the initial setup of a repository.

git status : To check the status of files you've changed in your working directory, i.e, what all has changed since your last commit.

git add filename : Add changes of specific file to stage/index in your working directory to be tracked.

git add . : Add changes of all files

GIT COMMANDS

Related with commits ...

git commit -m “commit msg” : to add or push new commit from staging area

-m should provide a meaningful Commit message. In our case we could use git commit -m "added starting code".

GIT COMMANDS

Related with commits ...

git log : in our terminal will display all Commits inside our Branch with details.

```
$ git log
commit c6ad406c1ffc84c40e88d5260e4f0cbd7b17bfa0 (HEAD -> master)
Author: Your Name <you@example.com>
Date:   Sun Jan 5 16:15:08 2020 +0200

    starting

commit c9fd3063d36cc605c1ef768715eccf308da05b3e
Author: Your Name <you@example.com>
Date:   Sun Jan 5 16:07:29 2020 +0200

    add starting html structure

commit ef64d62e328bcc2e2c73280d9818cfd27bbaefcf
Author: Your Name <you@example.com>
Date:   Wed Jan 1 11:40:06 2020 +0200

    Add header to blog
```

GIT COMMANDS

Related with commits ...

git log --oneline : will display all Commits without details with 7 first id key

```
$ git log --oneline
c6ad406 (HEAD -> master) starting
c9fd306 add starting html structure
ef64d62 Add header to blog
7b2fac6 Initial commit
```

GIT COMMANDS

Related with commits ...

git show : Shows the latest commit on your current branch.

git diff : command is used to see changes that have been made but haven't been committed, yet:

git revert <commit-id> : is used to undo a commit that reverses the changes from the specified commit — without changing the project's history.

GIT COMMANDS

Related with branch...

git branch : will list out the branches in a repository:

git branch asd :Will create a new branch called asd

git branch amir 7b9c5e2 : This creates a new branch amir starting from the commit 7b9c5e2.

git switch -c huda : -c is combo flag make you create branch and switch to it .

git switch asd :Switching to branch asd

git branch -m asd omar : rename branch asd to omar

git branch –d omar :Delete the branch omar

Merge branches...

For example you want to merge the branch of “omar” to branch of “master”

1. switch to master : **switch master**
2. merge omar on master **:** **merge omar**

Merge branches...

If you see conflict like this:

```
<!DOCTYPE html>
<html>

<head>changes</head>

Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes | Start Live Share Session
<<<<<<< HEAD (Current Change)
<body>
  <h1>Gitmas</h1>
  <p>This is the repository for my Gitmas blog post series!</p>
</body>
=======
<body>oh hai we made some changes</body>
>>>>>>> master (Incoming Change)

</html>
```

it's your choice how to solve it ,depends on what the code you need , there is no specific way to solve conflicts

Lab

GIT LAB

1. Download git , login
2. create your project demo and make demo file has operational functions (+,-)
3. make commit for each functions seperatly
4. create a branch with name “branch2” and by it make (/ , *) functions
5. create another branch with your name and switch to it , then:
    - make SayHello function
6. merge the branch which with your name to the main/master branch

note: each change you add , make it with a different commit.

Any Questions?